

Rust enum hint sheet

Valid Parentheses

Rewrite your stack solution using enums. Keep track of bracket kind and direction separately. Use the examples below, then complete the missing pieces.

1 Represent kind and direction

```
#[derive(Debug, PartialEq)]
enum Kind {
    Round,
    Square,
    Curly,
}
#[derive(Debug)]
enum Bracket {
    Open(Kind),
    Close(Kind),
}
```

An enum value is one of its named variants. `Open(Kind)` and `Close(Kind)` each carry a kind. `PartialEq` enables `==` and `!=`; `Debug` enables printing with `{:?}`.

2 Create a value and take it apart

```
let b = Bracket::Close(Kind::Square);
match b {
    Bracket::Open(kind) => println!("open {:?}" , kind),
    Bracket::Close(kind) => println!("close {:?}" , kind),
}
```

This prints `close Square`. `Kind` names the type; `kind` is a new variable holding the value extracted by the pattern. You could name that variable `k` instead.

3 Convert a character

Finish the four similar arms. `Option<T>` is a built-in enum: `Some(value)` holds a value; `None` means no value. In `Option<Bracket>`, that value is a `Bracket`.

```
fn to_bracket(c: char) -> Option<Bracket> {
    match c {
        '(' => Some(Bracket::Open(Kind::Round)),
        ')' => Some(Bracket::Close(Kind::Round)),
        '[' => todo!("opening square"),
        ']' => todo!("closing square"),
        '{' => todo!("opening curly"),
        '}' => todo!("closing curly"),
        _ => None,
    }
}
```

`_` matches any remaining character. `todo!()` marks unfinished code: it compiles, but panics if reached. Replace each one before testing that path.

Finish your stack solution

Put both enums and `to_bracket` above `impl Solution`. On LeetCode, keep its supplied `Solution` wrapper. The stack stores unmatched opening kinds. Fill in A, B, and C.

```
impl Solution {
    pub fn is_valid(s: String) -> bool {
        let mut stack: Vec<Kind> = Vec::new();
        for c in s.chars() {
            match to_bracket(c) {
                Some(Bracket::Open(kind)) => {
                    // A: Remember the opening kind.
                    todo!("A");
                }
                Some(Bracket::Close(kind)) => {
                    // B: Check the latest opening kind.
                    todo!("B");
                }
                None => return false,
            }
        }
        // C: Did every opening bracket get closed?
        todo!("C")
    }
}
```

Read `Some(Bracket::Close(kind))` from outside inward: a value exists, it is a closing bracket, and its stored kind becomes the variable `kind`.

4 Stack tools for A B and C

Expression	What it does
<code>stack.push(kind)</code>	Adds <code>kind</code> to the top of the stack.
<code>stack.pop()</code>	Removes the top item: <code>Some(kind)</code> , or <code>None</code> if empty.
<code>stack.is_empty()</code>	Returns <code>true</code> if no items remain.

5 Hints for the closing branch

Hint 1 Call `stack.pop()` once. Use `match` on its result: `Some(open)` gives you the latest opening kind; `None` means there was no opening to close.

Hint 2 Reject an empty stack or a different kind. If the kinds agree, continue the loop. Do not return `true` yet: more brackets may follow.

Hint 3 If you want a shorter check, compare the popped `Option<Kind>` with `Some(kind)`. What does inequality mean in each failure case?

For C, check for unmatched openings left in the stack. A final expression returns its value without a semicolon.

6 Check different failure cases

Input	Expected	What it checks
<code>"()[]{}"</code>	<code>true</code>	Separate pairs
<code>"{[]}"</code>	<code>true</code>	Nested pairs
<code>"[]"</code>	<code>false</code>	Wrong kind
<code>"([)]"</code>	<code>false</code>	Wrong closing order
<code>)"</code>	<code>false</code>	Closing without an opening
<code>"(("</code>	<code>false</code>	Openings left at the end